1. STRINGS:

A string is an ordered sequence of characters used to store text.

Strings are immutable in Python.

• Written in ' ' or " "

• Indexing ⟶ access characters

• Slicing ⟶ get part of string

```
s = "Vignan"

print(s[0])
print(s[-1])
print(s[1:4])
print(s[::-1])
```

```
V
n
ign
nangiV
```

other inbuilt functions :

```
name = "vignan"
print(name.upper())


print(name.lower())

line = "sai vignan a"
print(line.split())
```

```
VIGNAN
vignan
['sai', 'vignan', 'a']
```

```
n = "  sai vignan    "
print(n.strip())
```

```
sai vignan
```

```
word = "saiVignan"
print(word.find("V"))
```

```
3
```

```
code = "SaiVignan"
print(code[2:6])
```

```
iVig
```

```
print("123".isdigit())
print("abc".isalpha())
print("abc123".isalnum())
```

```
True
True
True
```

```
t = "name:Vignan:CS"
print(t.split(":"))
print(t.partition(":"))
```

```
['name', 'Vignan', 'CS']
('name', ':', 'Vignan:CS')
```

Tuples (())

A Tuple is like a list, but you cannot change it after it is created (immutable).

```
sai_tuple = ("Sai", "Vignan", "A")


print(sai_tuple[0])
```

```
Sai
```

Lists ([])

A List is an ordered collection that you can change (mutable)

```
lst = [10, "hi", 3.5, True]

print(lst[0])
print(lst[-1])
print(lst[1:3])
```

```
10
True
['hi', 3.5]
```

```
#to add any elements
nums = [1,2,3]
nums.append(4)
nums.insert(1, 1.5)
nums.extend([5,6])
print(nums)
```

```
[1, 1.5, 2, 3, 4, 5, 6]
```

```
sai_list = ["Sai", "Vignan", "A"]


sai_list.append("Student")
print(sai_list)
```

```
['Sai', 'Vignan', 'A', 'Student']
```

```
#to remove any :
nums.remove(1.5)
print(nums)

print(nums.pop())
print(nums.pop(1))
print(nums)
```

```
[1, 2, 3, 4, 5, 6]
6
2
[1, 3, 4, 5]
```

```
#other functions:
nums.reverse()
print(nums)

nums.sort()
print(nums)

nums.sort(reverse=True)
print(nums)
```

```
[5, 4, 3, 1]
[1, 3, 4, 5]
[5, 4, 3, 1]
```

** IF / ELIF / ELSE**

Conditional statements are used to execute code based on conditions.

• Only one block executes

• Conditions are checked top to bottom

• Indentation is mandatory

```
marks = 70

if marks < 40:
    print("Fail")
elif marks < 60:
    print("Average")
else:
    print("Pass")
```

```
Pass
```

FOR LOOP

A for loop is used to iterate over a sequence or range of values.

• Used when number of iterations is known

• Works with list, string, range

```
for i in [1,2,3]:
    print(i)
```

```
1
2
3
```

```
for ch in "Python":
    print(ch)
```

```
P
y
t
h
o
n
```

RANGE() and LEN()

range() generates a sequence of numbers, and len() returns the size of a sequence.

• range(stop) → 0 to stop-1

• Can use step & reverse

• Used for index-based loops

```
range(5)
range(1,6)
range(10,0,-2)
```

```
range(10, 0, -2)
```

```
langs = ["C","Python","Java"]

for i in range(len(langs)):
    print(i, langs[i])
```

```
0 C
1 Python
2 Java
```

```
Start coding or generate with AI.
```

WHILE LOOP

A while loop repeats code as long as a condition is True.

- Used when iterations are unknown
- Condition must change
- Risk of infinite loop

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
1
2
3
4
5
```

```
n = 5
sum = 0
i = 1
while i <= n:
    sum += i
    i += 1
print(sum)
```

```
15
```

```
n = 5
total = 0
i = 1

while i <= n:
    total += i
    i += 1

print(total)
```

```
15
```

BREAK & CONTINUE

break and continue control the flow of loops.

• break → exits loop

• continue → skips iteration

```
for i in range(5):
    if i == 3:
        break
    print(i)
```

```
0
1
2
```

```
for i in range(5):
    if i % 2 == 0:
        continue
    print(i)
```

```
1
3
```

Dictionaries ({})

A Dictionary stores data in Key-Value pairs. Your classwork mentions that objects use a special dictionary called **dict** to store their attributes.

```
#dictionary construction
sai_dict = {
    "first_name": "Sai",
    "last_name": "Vignan",
    "initial": "A",
    "role": "Python Developer"
}

print(sai_dict["first_name"])
```

```
Sai
```

```
my_dict = {'name': 'Vignan', 'age': 26}
```

```
my_dict['name']
my_dict.get('age')
```

```
26
```

```
nested = {'person': {'name': 'Vignan', 'age': 26}}
nested['person']['name']
```

```
'Vignan'
```

```
d = {'a': 1, 'b': 2}
print(d.keys())
print(d.values())
d.items()
```

```
dict_keys(['a', 'b'])
dict_values([1, 2])
dict_items([('a', 1), ('b', 2)])
```

```
#dict comprehension
{x: x**2 for x in range(1,6)}
```

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Copying Dictionaries

Shallow copy: references nested objects

Deep copy: copies everything recursively

```
import copy
deep_copy = copy.deepcopy(my_dict)
```

```
print(deep_copy.items())
```

```
dict_items([('name', 'Vignan'), ('age', 26)])
```

Functions

Functions allow reusable blocks of code.

```
def function_name(arg1, arg2):
    # body
    return arg1 + arg2

print(function_name(1, 2))
```

```
3
```

```
def greet(name):
    print(f"Hello {name}")

greet("Vignan")
```

```
Hello Vignan
```

```
def func_print():
    print("Hi")  # returns None

def func_return():
    return "vignan"  # returns string

a = func_print()
b = print(func_return())
```

```
Hi
vignan
```

```
name = ["sai", "vignan", "a"]
last = name.pop()

print(name)
```

```
['sai', 'vignan']
```

```
name = ["sai", "vignan", "a"]
last = name.remove("sai")

print(name)
```

```
['vignan', 'a']
```

Iterators and Generators

Iterator: it is an object 'iter()' throgh which the parameter/variables changes into a iteratable object, and we can traverse using next(),(moves forward, and cant go any furthur than the length of the value).

Generator: smart iterator that remembers the previously yielded value when called again. (memory-efficient, useful in loops). uses function 'yield()'.

```
#iterator

name = "Vignan"

it = iter(name)

print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

```
V
i
g
n
a
n
---------------------------------------------------------------------------
StopIteration                             Traceback (most recent call last)
/tmp/ipython-input-2320724426.py in <cell line: 0>()
     11 print(next(it))
     12 print(next(it))
---> 13 print(next(it))
     14 print(next(it))

StopIteration:
```

Next steps: Explain error

```
#generator

def squares(n):
    for i in range(n):
        yield i * i

g = squares(5)

for x in g:
    print(x)

# 0, 1, 4, 9, 16
```

```
0
1
4
9
16
```

```
def gencubes(n):
    for i in range(n):
        yield i**3

for x in gencubes(5):
    print(x)
```

```
0
1
8
27
64
```

```
def spell(name):
    for ch in name:
        yield ch

g = spell("Sai Vignan")

for letter in g:
    print(letter)
```

```
S
a
i

V
i
g
n
a
n
```

map

Takes a function and applies it to every item in an iterable.

Basically a hidden for-loop that transforms data.

Returns a map object, so I need to wrap it in list() to see results.

Great when I want clean functional-style code without writing loops.

```
temps_c = [0, 10, 25]

def c_to_f(c):
    return (c * 9/5) + 32

result = list(map(c_to_f, temps_c))
print(result)
```

```
[32.0, 50.0, 77.0]
```

```
tasks = ["clean", "study", "cook"]

def mark_done(t):
    return t + "_done"

result = list(map(mark_done, tasks))
print(result)
```

```
['clean_done', 'study_done', 'cook_done']
```

REDUCE

Repeatedly applies a function to items and collapses them into one value.

Works left to right: takes first two items, reduces them, then uses result with the next item.

Good for things like summing, multiplying, finding max, combining strings.

Needs from functools import reduce.

```
# This is formatted as code
```

```
#reduce

from functools import reduce

words = ["Sai", "Vignan", "is", "learning", "Python"]

def combine(a, b):
    return a + " " + b

result = reduce(combine, words)
print(result)
```

```
Sai Vignan is learning Python
```

```
from functools import reduce

nums = [2, 3, 4]

def multiply(a, b):
    return a * b

result = reduce(multiply, nums)
print(result)
```

```
24
```

LAMBDA

A tiny unnamed function I can write in one line.

Perfect for small throwaway logic I don't want to define with def.

Only one expression inside; no statements allowed.

Usually paired with map, filter, reduce.

```
#lambda

last_digit = lambda n: n % 10

print(last_digit(9872))
```

```
2
```

```
format_name = lambda name: name.strip()

print(format_name("  sai vignan  "))
```

```
sai vignan
```

FILTER

Takes a function and keeps only the items for which the function returns True.

Basically a hidden for-loop that throws away unwanted items.

Returns a filter object, so I need to wrap it in list() to view the output.

Perfect for selecting, cleaning, or removing things from data.

Works great with simple functions or lambdas when the condition is short.

```
#filter

names = ["Sai", "Vignan", "Ayancha"]

def long_name(n):
    return len(n) > 5

result = list(filter(long_name, names))
print(result)
```

```
['Vignan', 'Ayancha']
```

```
nums = [5, -2, 9, -7, 0, 3]

def is_negative(x):
    return x < 0

result = list(filter(is_negative, nums))
print(result)
```

```
[-2, -7]
```

File Handling

File handling allows you to store data permanently on your hard drive rather than keeping it in temporary memory. Using the with statement is the best practice because it ensures the file closes automatically after you are done writing or reading.

used for ETL.

```
with open("sai_profile.txt", "w") as f:
    f.write("Name: Sai Vignan A\nRole: Python Developer")


with open("sai_profile.txt", "r") as f:
    print(f.read())
```

```
Name: Sai Vignan A
Role: Python Developer
```

```
with open("sai_profile.txt", "r") as src, open("backup_notes.txt", "w")
    for line in src:
        dest.write(line)
with open("backup_notes.txt", "r") as f:
    print(f.read())
```

```
Name: Sai Vignan A
Role: Python Developer
```

```
with open("sai_profile.txt", "r") as f:
    first_20 = f.read(20)

print("First 20 characters:", first_20)
```

```
First 20 characters: Name: Sai Vignan A
R
```

```
#gives both reading and writing ability.

with open("sai_profile.txt", "r+") as f:
    content = f.read()
    f.seek(20)           # move cursor to 20th character
    f.write("INSERTED_TEXT")
with open("sai_profile.txt", "r") as f:
    print(f.read())
```

```
Name: Sai Vignan A
RINSERTED_TEXTeveloper
```

NumPy (np)

NumPy is a powerful library used for numerical calculations and managing large, multi-dimensional arrays. It is much faster than standard Python lists because it performs "vectorized" operations on entire blocks of data at once.

```
Start coding or generate with AI.
```

```
import numpy as np

sai_scores = np.array([85, 90, 88, 92])


final_scores = sai_scores + 5
print(f"Original: {sai_scores}")
print(f"With Bonus: {final_scores}")
```

```
Original: [85 90 88 92]
With Bonus: [90 95 93 97]
```

Pandas(pd):

Pandas is a Python library used to work with structured data like tables.(excel).

```
import pandas as pd

data = {
    "Student": ["Liam", "Emma", "Noah", "Olivia", "Ava"],
    "Score": [92, 85, 78, 88, 95]
}

df = pd.DataFrame(data)
print(df)
```

```
  Student  Score
0    Liam     92
1    Emma     85
2    Noah     78
3  Olivia     88
4     Ava     95
```

```
import pandas as pd

data = {
    "Employee": ["Arjun", "Meera", "Rahul", "Sneha"],
    "Salary": [70000, 82000, 65000, 90000]
}

df = pd.DataFrame(data)
print(df)
```

```
  Employee  Salary
0    Arjun   70000
1    Meera   82000
2    Rahul   65000
3    Sneha   90000
```

```
#loading this tabular data into an excel file,
#DataFrame is like an Excel table in Python.
#Columns represent fields, rows represent records.

df.to_excel("employees.xlsx", index=False) #putting index false, will
```

Always use with open() for safety.

"w" deletes old data, "a" preserves it.

read() → whole file

readline() → one line

readlines() → list of lines

Pandas is required for Excel file handling.

```
Start coding or generate with AI.
```

modules

they are jsut another .py files which we or someone creates, having some functions defined in them, which we can use by importing that module into our current .py file, just like transfering somebody else functions and makeing use of them.

we have some predefined/ inbuilt modules in python to make our day easy.

some pre-installed/ inbuilt modules :

```
import math
print(math.sqrt(49))
```

```
7.0
```

```
from math import factorial # importing specific funciton
print(factorial(6))
```

```
720
```

```
import random
print(random.randint(100, 200))
```

```
157
```

```
import math
print(dir(math)) # we can see what kind of functions, variables etc p
```

```
['__doc__', '__loader__', '__name__', '__package__', '__spec__', 'acos
```

```
import math
help(math.pow)
```

```
Help on built-in function pow in module math:

pow(x, y, /)
    Return x**y (x to the power of y).
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
print(10 / 0)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
/tmp/ipython-input-1505361072.py in <cell line: 0>()
----> 1 print(10 / 0)

ZeroDivisionError: division by zero
```

Next steps: Explain error

ERRORS/EXCEPTION HANDLING

some errors which can be expected to happen which occur at run time, can be stopped or skipped using this exception handling, as the name suggest exceptions.

```
int("abc")        # ValueError
5 / 0             # ZeroDivisionError
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipython-input-3331306372.py in <cell line: 0>()
----> 1 int("abc")        # ValueError
      2 5 / 0             # ZeroDivisionError

ValueError: invalid literal for int() with base 10: 'abc'
```

Next steps: Explain error

try-except blocks

using a try block, we can write code, which might cause any exceptional errors and catch them from stopping/crashing the entire program run. once the code in the try block does not look to be running, python will go on to the exception block, which handles the errors which are caused in the try block.

```
try:
    data = open()
except:
    print("Some error occurred")
```

```
Some error occurred
```

```
try:
    result = 100 / 0
except ZeroDivisionError:  # python inbuilt exception block
    print("Cannot divide by zero")
```

```
Cannot divide by zero
```

```
try:
    x = int("hello")
    y = 10 / 0
except ValueError:
    print("Invalid number conversion"). #pyton inbuilt exceptiojn blo
except ZeroDivisionError:
    print("Division error")
```

```
Invalid number conversion
```

```
try:
    num = int("42")
except:
    print("Conversion failed")
else:      # if no exceptions occured..
    print("Converted successfully:", num)
```

```
Converted successfully: 42
```

**finally block **

this block will run despite error occurs or not. can be used at the end.

Double-click (or enter) to edit

```
try:
    f = open("abcd.txt", "w")
    f.write("Exception handling notes")
except:
    print("Write failed")
finally:
    f.close()
    print("File closed safely")
```

```
File closed safely
```

OOPS

OOP is a set of core ideas that define how objects are created, used, and interact.

OOP mainly consists of classes, objects, and few core principles. These ideas help in writing reusable, structured, and maintainable code.

Classes & Objects

Think of a Class as the blueprint and the Object as the actual thing you build from it.

init: This is the "initializer." It runs automatically to set up your data when you create a new object.

self: This represents the specific object you are working with right now.

```
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no

# object creation
sai = Student("Sai Vignan A", 101)
print(f"Name: {sai.name}, ID: {sai.roll_no}")
```

```
Name: Sai Vignan A, ID: 101
```

```
Start coding or generate with AI.
```

Methods and **str.**

Methods are just functions that live inside a class.

str: If you try to print() an object, it usually looks like gibberish. **str** lets you choose a nice message to show instead.

```
class Profile:
    def __init__(self, name):
        self.name = name

    def __str__(self):
        return f"This profile belongs to {self.name}"

me = Profile("Sai Vignan A")
print(me)
```

```
This profile belongs to Sai Vignan A
```

```
Start coding or generate with AI.
```

Abstraction and **(Encapsulation)**

abstraction is the concept of only exposeing those details which need to be, and hiding others.

encapsulation is the concept of hiding the code or methods as well from others to access to, instead they can access them indirectly.

In Python, we don't have locks for data, but we use underscores to tell other coders to not touch this directly.OOP lets us hide internal details so they don't get messed up by accident.

_variable: Internal/Private (suggested).

__variable: Strongly private (Python mangles the name so it's harder to find).

Single Underscore (_variable): This is the "Protected" style. It tells others, "I'm using this for internal stuff, please don't change it directly," but Python won't actually stop you from accessing it.

Double Underscore (__variable): This is the "Private" style. Python performs "name mangling" here, making it much harder to access from outside the class to keep the data safe.

```
class Laptop:
    def __init__(self, owner):
        self.owner = owner
        self.__password = "Sai_Secure_123" # Private

my_pc = Laptop("Sai Vignan A")
print(my_pc.owner)
# print(my_pc.__password)
```

```
Sai Vignan A
```

```
class Laptop:
    def __init__(self, owner):
        self.owner = owner          # Public
        self._os = "Windows 11"     # Protected
        self.__password = "Sai_123" # Private


my_pc = Laptop("Sai Vignan A")

print(my_pc.owner)    # Works
print(my_pc._os)      # Works, But Bad practice to access
# print(my_pc.__password) ### CRASHES
```

```
Sai Vignan A
Windows 11
```

```
#encapsulation :

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance   # private variable

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount

    def get_balance(self):
        return self.__balance


acc = BankAccount("Vignan", 1000)

acc.deposit(500)
acc.withdraw(300)

print(acc.get_balance())
```

```
1200
```

Inheritance

This is when a "Child" class takes all the features of a "Parent" class so you don't have to rewrite code.

super(): This is a shortcut to call the functions from the Parent class.

```
class Person:
    def __init__(self, name):
        self.name = name

class Coder(Person): # Coder inherits from Person
    def __init__(self, name, language):
        super().__init__(name) # Run the Person init
        self.language = language

sai = Coder("Sai Vignan A", "Python")
print(f"{sai.name} codes in {sai.language}")
```

```
Sai Vignan A codes in Python
```

Polymorphism and Duck Typing

Polymorphism means the same function or method can be used or behave differently depending on the object that is using it.(different object, diff paramters-->> diff output)

Polymorphism (Duck Typing)

bold text"If it walks like a duck and quacks like a duck, it's a duck." In Python, as long as two different objects have a method with the same name, you can use them the same way.

Polymorphism with a Function

Imagine you and a friend are both "Workers," but you do different things. We can write one function that handles both of you as long as you both have a work() method.

```
class Dog:
    def speak(self): return "Woof!"

class Cat:
    def speak(self): return "Meow!"

def make_it_talk(animal):
    print(animal.speak())

sai_dog = Dog()
sai_cat = Cat()

make_it_talk(sai_dog) # Works
make_it_talk(sai_cat) # Also works
```

```
Woof!
Meow!
```

```
class SaiVignan:
    def work(self):
        return "Coding in Python"

class AI:
    def work(self):
        return "Processing data"

# This function is polymorphic - it accepts ANY object with a .work()
def start_task(entity):
    print(entity.work())

sai = SaiVignan()
bot = AI()

start_task(sai)
start_task(bot)
```

```
Coding in Python
Processing data
```

```
#overriding – child class inherits methods from the superclass, but cl
class Payment:
    def pay(self, amount):
        print("Child must implement pay()")

class CreditCardPayment(Payment):
    def pay(self, amount):
        print(f"Charging ${amount} to credit card.")

class PayPalPayment(Payment):
    def pay(self, amount):
        print(f"Charging ${amount} via PayPal.")
    # pass

payments = [CreditCardPayment(), PayPalPayment()]
for p in payments:
    p.pay(100)
```

```
Charging $100 to credit card.
Charging $100 via PayPal.
```

Dynamic Extension/Composition (Wrapping)

This is another way to build classes. Instead of "inheriting," one class just contains another.

Example: A Student has a Brain.

in inheritance, everything all the functions are borrowed to the child class from parent class.

in wrapping, the child only takes the functions from the super/parent class whats needed, not everything. this way code is more capsulated.

```
class Brain:
    def think(self):
        return "Thinking about Python..."

class Student:
    def __init__(self, name):
        self.name = name
        self.brain = Brain() # Putting a Brain object inside Student

sai = Student("Sai Vignan A")
print(sai.brain.think())
```

```
Thinking about Python...
```

```
class Calculator:
    def add(self, x, y):
        return x + y

# WRAPPING: Sai's class HAS a Calculator inside it
class SaiWork:
    def __init__(self):
        self.c = Calculator() # Wrapping the Calculator

    def do_math(self, a, b):
        return self.c.add(a, b) # Telling the internal object to work

sai = SaiWork()
print(sai.do_math(10, 20)) # You access the math through 'sai'
```

```
30
```

```
Start coding or generate with AI.
```

DATA class and post **init**

Data Class (@dataclass): A decorator that automatically generates the **init**, **repr**, and **str** methods so you don't have to write repetitive assignment code.

post_init: A method that runs automatically after the data class assigns its initial values, used for performing extra logic like data validation or calculations.

```
from dataclasses import dataclass

@dataclass
class StudentRecord:
    # --- Data Class Fields ---
    # Python creates __init__(self, name, marks) automatically
    name: str
    marks: int
    bonus: int = 5
    final_score: int = 0  # This will be calculated in post_init

    def __post_init__(self):
        #  Validation: Ensure marks are within range
        if self.marks < 0 or self.marks > 100:
            raise ValueError("Marks must be between 0 and 100")


        # Computation: Calculate the final score
        self.final_score = self.marks + self.bonus


student = StudentRecord(name="sai vignan", marks=85)

print(student)
```

```
StudentRecord(name='sai vignan', marks=85, bonus=5, final_score=90)
```

Class vs Static Methods:

@classmethod: Used when you need to access "Class" level data (shared by everyone), not just one person's data.

@staticmethod: Just a normal function that lives inside the class because it's related, but it doesn't need any data from the class or object.

```
Start coding or generate with AI.
```

Class Methods (@classmethod)

A class method belongs to the Class itself, not a specific object. It uses cls instead of self. If you change something using a class method, it changes for every object made from that class.

Example: Keeping track of how many students are in a school.

```
class Student:
    total_students = 0 # Class variable

    def __init__(self, name):
        self.name = name
        Student.total_students += 1

    @classmethod
    def get_school_size(cls):
        # 'cls' refers to the Student class
        return f"Current students: {cls.total_students}"

sai = Student("Sai Vignan A")
friend = Student("Rahul")

print(Student.get_school_size()) # Output: Current students: 2
```

```
Current students: 2
```

```
Start coding or generate with AI.
```

Static Methods (@staticmethod)

A static method is like a "guest" living inside the class. It doesn't use self or cls. It’s just a regular function that you put inside a class because it's logically related to it, but it doesn't need to change any student data.

Example: A simple calculator inside the Student class to help with homework.

```
class Student:
    @staticmethod
    def add_marks(m1, m2):
        # Doesn't need name, ID, or school data
        return m1 + m2

# You don't even need to create 'sai' to use this!
print(Student.add_marks(80, 90))
```

```
170
```

```
class Student:
    school_name = "ABC School"  # Class Variable: Shared by everyone
    total_students = 0          # Class Variable: Shared by everyone

    def __init__(self, name, marks):
        self.name = name        # Instance Variable: Unique to Sai
        self.marks = marks      # Instance Variable: Unique to Sai
        Student.total_students += 1

    #INSTANCE METHOD (Normal)
    #Needs 'self' to access personal data like 'self.name'
    def get_personal_report(self):
        return f"Student {self.name} scored {self.marks}."

    #CLASS METHOD
    #Needs 'cls' to access/change Class data like 'school_name'
    @classmethod
    def change_school(cls, new_name):
        # cls.school_name = new_name
        return f"School updated to {cls.school_name}"

    #STATIC METHOD
    # No 'self' or 'cls'. Just a regular fxn kept inside the class
    @staticmethod
    def calculate_grade(score):
        return "Pass" if score >= 40 else "Fail"

#testing...

# Create instance
sai = Student("Sai Vignan A", 85)
```

```
# A. Call Instance Method: It knows 'self' is Sai
print(sai.get_personal_report())

# B. Call Class Method: It updates the school for EVERYONE
print(Student.change_school("XYZ University"))

# C. Call Static Method: It doesn't care about Sai or the School
print(Student.calculate_grade(85))
```

```
Student Sai Vignan A scored 85.
School updated to ABC School
Pass
```

Start coding or generate with AI.